Task 6.1 - Model Version Inventory & Comparison of Performance

Version	Model Identifier / Family	Key Configuration	Test MAE	Test RMSE	Test R2	Status
v1.0-baseline	LinearRegression Family : Ordinary Least Squares	Default fit intercept, 21 engineered features	844.86	1,081.68	0.7027	Retired (Benchmark)
v1.1-tree	RandomForestRegressor Family : Bagged Decision Tree Ensemble	n_estimators=100, max_depth=18, min_samples_split=5, random_state=42	**296.62**	**511.91**	**0.9334**	**Best (Production)**
v2.0-deep	TrafficLSTM Family : 2-Layer Recurrent Neural Network	seq_len=24, hidden_dim=64, num_layers=2, dropout=0.2, Adam (lr=1e-3)	355.05	551.61	0.9227	Contender (Next best candidate)

Versioning and Selection Details

1. **v1.0-baseline (Linear Regression)**: Started with a basic linear model using cyclic sine and cosine time features. An average error of 845 vehicles per hour showed that traffic patterns are far too complex and non-linear for a simple straight-line model.
2. **v1.1-tree (Random Forest – Best and selected)**: Best overall performance, explaining 93.3% of the variance ($R^2 = 0.9334$) and cutting the error down to roughly 297 vehicles per hour. Because it delivers top accuracy and runs fast predictions without heavy hardware, it was the clear choice for production and hence selected.
3. **v2.0-deep (PyTorch LSTM – Next best, contender)**: Modeled 24-hour sequence trends well and came close in performance ($R^2 = 0.9227$, 355 vehicles/hr error). However, it requires more computation power to train and serve without beating Random Forest, so it remains a second best candidate.

Task 6.2: Experiment Tracking with Mlflow

1. Setup and Workflow
    - A unified experiment was organised to have all runs under a single experiment titled “SmartCity_Traffic_Volume_Forecasting” for easily comparing models side by side.
    - Instead of relying on local files, a local database (SQLite database) was used, and MLFlow was pointed to that database. This helped to keep the experiment history stable, consistent and easy to inspect across multiple runs

2. What was tracked across models
    - An automated tracking script (track_experiments_mlflow.py) was used to log parameters, evaluation scores, and saved model binaries for the 3 iterations.
    - Baseline Linear Regression:
        - Parameters: 21 input features, standard intercept fit.

        - Results: MAE of 844.86, RMSE of 1,081.68, and an R^2 of 0.7027.
        - Artifacts: Saved the Scikit-learn model and environment details under model/.
    - Random Forest Regressor (Champion):
        - Parameters: 100 trees (n_estimators=100), maximum tree depth of 18, and minimum split of 5.
        - Results: MAE dropped to 296.62, RMSE improved to 511.91, and R^2 reached 0.9334.
        - Artifacts: Logged the trained ensemble binary along with the feature list.
    - PyTorch 2-Layer LSTM (Deep Learning Challenger):
        - Parameters: 24-step sequence lookback, 64 hidden units, 2 layers, 20% dropout, Adam optimizer with a 0.001 learning rate.
        - Results: MAE of 355.05, RMSE of 551.61, and an R^2 of 0.9227.
        - Artifacts: Logged the PyTorch model artifact along with a sample tensor input to record the expected input signature.

3. Dashboard Review
    - MLFlow dashboard was launched locally on port 5001.
    - The logged runs were verified in the MLFlow dashboard
    - The dashboard confirmed that the 3 runs completed successfully without errors and the model artifacts were stored properly.
    - Comparison of the metric charts, side by side, confirmed the earlier finding that Random Forest model offered the best balance of low prediction error, and simple computational requirements.

Task 6.3: Model Deployment Mock-up (FastAPI)

1. Setup and Design
    - Web Service: FastAPI and Uvicorn (deployment/app.py) were used to create a simple web service that serves model predictions.
    - The Random Forest model (best_regression_rf.pkl) loads once when the app starts up so incoming requests get scored right away without lag.
    - Pydantic was used to make sure inputs are clean and valid (e.g., hours stay between 0 and 23, clouds stay between 0% and 100%).
2. Endpoints
    - GET /health: A simple check to confirm the service is running, which model is loaded, and the expected number of features.
    - POST /predict:
        - Takes in current conditions: time, day of week, temperature, rain, clouds, and weather type.
        - Formats these inputs into the 21 features our model needs (like cyclical sine/cosine time values and weather categories).
        - Returns the predicted vehicles per hour.
3. Testing the Endpoint
    - A test request was sent using curl:

```
curl -X POST http://127.0.0.1:8000/predict \
      -H "Content-Type: application/json" \
      -d '{"hour": 8, "day_of_week": 0, "temp_c": 15.0, "rain_1h":
      0.0, "clouds_all": 20, "weather_main": "Clear"}'
```

- Returned Output:

```
      JSON
      {
        "status": "success",
        "model_version": "v1.1-RandomForestRegressor",
        "hour": 8,
        "predicted_traffic_volume": 5595.4,
        "unit": "vehicles/hour"
      }
```

The API returned a prediction of roughly 5,595 vehicles per hour for Monday morning peak traffic in under 10 milliseconds.

Task 6.4: Production Model & Data Drift Monitoring

1. Checking for Data Shifts: Traffic patterns and weather change across seasons. If live conditions look completely different from what the model saw during training, its predictions can become unreliable.
2. What was tracked:
    - Input Features: Weather variables like temperature and cloud cover was monitored to see if incoming conditions are drifting.
    - Model Outputs: Predicted traffic volumes was monitored to catch unexpected spikes or drops.
3. Statistical Test: The two-sample Kolmogorov-Smirnov (KS) test was used with a cutoff of $p < 0.01$. If p drops below 0.01, it means the live data distribution differs significantly from the training baseline rather than just normal random noise.
4. The drift checks were run using the Python script part3_machine_learning/monitoring/monitor_drift.py, utilizing scipy.stats (specifically ks_2samp), pandas, and numpy to compare live batch samples against the baseline dataset (traffic_features.csv).

Simulation Results

Window 1: Standard Weekday Operations - Simulation of normal weekday traffic and seasonal weather:

Monitored Variable	Training Mean	Window 1 Mean	KS Statistic	p-value	Drift Result
Temperature (°C)	8.11	8.04	0.1021	**3.70×10^(-3)**	Minor Variance
Cloud Cover (%)	49.37	46.67	0.0406	0.695	NO (Normal)

Traffic Volume	3,259.62	3,319.76	0.0408	0.688	NO (Normal)

Takeaway: Both cloud cover and predicted traffic volumes match the baseline data closely ($p > 0.68$). The model behaves normally under expected day-to-day conditions.

Window 2: Severe Winter Storm & Congestion Spike - Simulation of extreme sub-zero blizzard with heavy traffic buildup:

Monitored Variable	Training Mean	Window 2 Mean	KS Statistic	p-value	Drift Result
Temperature (°C)	8.11	-22.24	0.9333	0.00	YES (Severe Drift)
Cloud Cover (%)	49.37	92.18	0.6859	6.07×10^(-140)	YES (Severe Drift)
Traffic Volume	3,259.62	6,093.88	0.8198	2.73×10^(-220)	YES (Severe Drift)

Takeaway: The severe cold (-22.24°C) and near-total overcast (92.18%) diverge sharply from training data. This causes predicted traffic volume to jump to $6{,}093.88$ vehicles per hour. The test flags both input feature drift and output prediction drift.

Task 6.5: Alerting Mechanism & Operational Health Dashboard

1. Alerting Design & Logic
    - Objective: Translate statistical drift tests into a simple operational status: PASS / Normal or ALERT / Requires investigation.
    - The alerting engine was implemented in `part3_machine_learning/monitoring/alerting_system.py`, which evaluates drift metrics from `monitor_drift.py` and returns the final operational health status.
    - Two-Stage Rule:
        - Statistical Significance ($p < 0.01$): Confirms that differences between incoming data and baseline data are not random noise.
        - Shift Size Threshold (KS > 0.25): Ensures alerts only fire when changes are practically meaningful, preventing false alarms from minor shifts.
    - System Action:
        - PASS / Normal: Confirms data stability and allows model scoring to run without intervention.
        - ALERT / Requires investigation: Flags anomalous batches, notifies operators, and requests model retraining on the new pattern.
2. Test Scenarios
   Scenario A: Standard Operating Window
    - Evaluated Batch: Batch Run: Weekday Standard Operations
    - System State: [PASS / Normal]

- Results:
  - Temperature: Baseline 8.11°C vs Batch 8.04°C (KS $= 0.1021$, Status: HEALTHY)
  - Cloud Cover: Baseline 49.37% vs Batch 46.67% (KS $= 0.0406$, Status: HEALTHY)
  - Predicted Traffic Volume: Baseline $3{,}259.62$ vs Batch $3{,}319.76$ veh/h (KS $= 0.0408$, Status: HEALTHY)
  - Action: [AUTOMATED SIGN-OFF] — Normal scoring continues without operator intervention.

Scenario B: Environmental Anomaly Window

- Evaluated Batch: Batch Run: Arctic Blizzard Influx
- System State: [ALERT / Requires investigation]
- Results:
  - Temperature: Baseline 8.11°C vs Batch -22.24°C (KS $= 0.9333, p = 0.00$, Status: CRITICAL DRIFT)
  - Cloud Cover: Baseline 49.37% vs Batch 92.18% (KS $= 0.6859, p = 6.07 \times 10^{-140}$, Status: CRITICAL DRIFT)
  - Predicted Traffic Volume: Baseline $3{,}259.62$ vs Batch $6{,}093.88$ veh/h (KS $= 0.8198, p = 2.73 \times 10^{-220}$, Status: CRITICAL DRIFT)
  - Action: [ESCALATION TRIGGERED] — Operations team is notified, output confidence flags are set, and retraining on the extreme weather regime is scheduled.